在虚拟机上复现 LEBench

Linux 核心操作性能演化分析

邹浩宇　5260000157
CS340 操作系统，2026–2027

2026 年 11 月 3 日

本文与英文版 `report/report.tex` 使用同一批客户机日志。表中的测试名与 CSV 中的名字一致，其中上下文切换在作者代码里写成 `context siwtch`。

1　引言与论断

Ren 等人测量了 Linux 3.0 到 4.20 上核心操作的延迟，认为内核变慢了，主要原因是安全补丁、新功能，以及少量配置错误 [1]。这些操作就是 LEBench 直接计时的项目：`getpid`、上下文切换、fork、读/写、mmap/munmap、缺页、send/recv，以及 select/poll/epoll。

论文里有三条论断：

1. **图 1(a)**。相对 Linux 4.0，更晚的内核在这些操作上更慢，大块写和大块 munmap 除外。多数测试也比 3.0 更慢，而且很多至少慢 50%。
2. **表 3 与图 2**。十一个改动解释了大幅跳动。其中四个是安全补丁，四个是新功能，三个是配置错误。
3. **第 5 节**。关掉这十一项之后，Redis、Apache 和 Nginx 最多分别加快 56%、33% 和 34%。

这次复现检验作者实验笔记里那份内核清单上的论断 1，并把论断 2 里的一个因素（页表隔离）当作扩展实验。论断 3 没有跑。作者的 `EXPERIMENTS.md` 列出了 41 个 Ubuntu 主线版本，从 3.0.7 到 4.20.13。论文正文写的是 36 个版本。驱动程序对这份清单里的每一个版本都做了尝试。

2　复现设置

2.1　起点代码

基准程序是作者的 LEBench，提交 `6aa7801`（`https://github.com/LinuxPerfStudy/LEBench`），Apache-2.0，版权 2019 年多伦多大学。`TEST_DIR/OS_Eval.c`、`run.py` 和 `get_kern.py` 没有改。课

程用的运行程序是 course/OS_Eval_reduced.c，由 course/make_reduced_eval.py 从原文件生成。它保留同样的 clock_gettime 区间和同样的 K-best 规则（五个样本落在 5% 以内）。BASE_ITER 从 10,000 改成 40，去掉了 60 秒的静置，并且不编译三项会阻塞或占满运行时间的测试：96 KB 的 send/recv、12,000 页的 fork，以及 10,000 页的文件测试。在嘈杂的虚拟机上，原程序遇到空的 K-best 结果会退出；这里改成退回到最小的那次样本。

另一份用 rdtsc 重写测试的代码没有用来出数。原因见第 6 节。

2.2 环境

项目	设置
宿主机	Windows 11，第 13 代 Intel Core i7-13700H
虚拟机	VirtualBox 7.2.22，传统 BIOS，NAT
客户机	Ubuntu 16.04.6 LTS，英文
资源	2 个虚拟 CPU，4 GB 内存，80 GB 磁盘，位于 D:
编译器	gcc 5.4.0（-O0 -pthread，与上游 makefile 相同）
恢复用内核	4.4.0-210-generic，只用来安装和恢复

宿主机开了内存完整性，VirtualBox 不能走它的快速虚拟化路径。客户机看到的 CPU 型号字符串仍然是宿主机的 i7-13700H。这颗 CPU 对 Meltdown 的报告是“Not affected”。

2.3 工作负载

每个启动成功的内核都以 root 运行三次 course/sweep_bench。这个二进制文件就是精简后的 OS_Eval。它写出作者原来的 CSV：每个测试一行 K-best、一行平均值，单位是秒。报告里的数字是三次 K-best 的均值和样本标准差。日志在 results/kernels/< 版本 >/runN.csv，另外还有标准输出、标准错误、退出码和 environment.txt。

内核包来自作者的备份仓库 https://github.com/LinuxPerfStudy/ExperimentSetup，目录 default_ubuntu_kernels。驱动程序下载 linux-image 的 deb（如果只有 unsigned 包，就用 unsigned），装上之后用 grub-reboot 指定下一次启动。如果 panic，下一次开机回到 4.4.0-210。没有对分区执行上游的 grub-install。

2.4 与论文的差别

项目	论文	本次	原因
机器	HP DL160 G9，Xeon E5-2630 v3，128 GB，SSD	i7-13700H 上的 VirtualBox，2 个 vCPU，4 GB	没有那台服务器。比较只在同一台客户机内进行。
内核数量	正文写 36 个版本	作者列出的全部 41 个版本	发布出来的清单才是可以复现的集合。
迭代次数	`BASE_ITER` 为 10,000，并静置 60 秒	`BASE_ITER` 为 40，不静置	在这台客户机上，完整一轮要几个小时。K-best 规则没有改。
省掉的测试	96 KB 的 send/recv、12,000 页的 fork、10,000 页的 I/O	不编译进去	上游注释写明大块 send 在旧内核上会阻塞。另外两项占满运行时间。
统计量	进程内的 K-best	先取该值，再对 3 个进程做均值 ± 样本标准差	课程要求报告跨次运行的统计量。
应用程序	Redis、Apache、Nginx	没有跑	先做论断 1 和一项安全开关。

3 结果

3.1 已安装内核上的页表隔离

这次对照只改一个因素。内核 4.4.0-210-generic 启动两次：一次用默认命令行，一次在同一内核上加上 `nopti`。硬件、二进制和迭代次数都不变。两次启动里，`/sys/devices/system/cpu/vulnerabilities/meltdown` 都是“Not affected”。Spectre v2 保持 Enhanced IBRS。

表 1 由 `scripts/render_results.py` 根据 `results/kernel_summary.csv` 算出。“重叠”表示两个“均值 ± 标准差”区间相交。

快的操作，变化幅度小于各次运行之间的波动。`fork` 在 `nopti` 下更慢，而且区间不重叠。这和论文里的 KPTI 开销方向相反，也不是一次干净的消融：这颗 CPU 本来就没有因为 Meltdown 打开页表隔离。

3.2 内核扫描

`EXPERIMENTS.md` 里的 41 个版本中，34 个启动到可以完成三次运行。3.0.7 到 3.5.4 能装上，但 sshd 始终没有应答；它们记在 `results/kernels/boot-failures.txt` 里，没有延迟数据。Linux 4.19.11 还需要一个 `linux-modules` 包，给 4.16、4.17、4.18 和 4.20 用的那个 Ubuntu 主线地址上已经没有这个包，所以这一版没有安装。4.4.0-210 这一对是客户机的恢复内核，不属于那 41 个版本。

比 4.0.5 更晚、并且完成了三次运行的内核共有 19 个，合计 627 项测量。其中 271 项（43%）

表 1: 同一内核、同一二进制。默认命令行对照 `nopti`。各三次。

测试	默认	`nopti`	变化	区间重叠
getpid	314 ns	295 ns	-6%	是
context siwtch	11.2 us	12.9 us	+15%	是
send	600 ns	693 ns	+15%	是
recv	692 ns	764 ns	+10%	是
fork	92.8 us	254.0 us	+174%	否
small read	784 ns	946 ns	+21%	是
small write	1.9 us	2.0 us	+7%	是
big write	1.66 ms	3.62 ms	+118%	是
big munmap	269.5 us	266.3 us	-1%	是
select	1.2 us	1.5 us	+29%	是
poll	847 ns	1.1 us	+26%	是
epoll	905 ns	1.0 us	+12%	是

的均值高于同一测试在 4.0.5 上的均值，76 项（12%）的均值至少高 50%。117 项的“均值 ± 标准差”区间与 4.0.5 不重叠，并且均值更高。分析里把这一条当作“确实更慢”的标准。

3.6.10 上 getpid 和另外几项的 K-best 是 0。这是一次失败的稳定窗口：同一份 CSV 里的平均值并不是 0。3.6.10 也不是基线，所以这几项不在那 627 项里面。表 2 给出三项测试。每个均值都是该版本目录下 `run1.csv`、`run2.csv` 和 `run3.csv` 里三次 K-best 的均值，全表在 `results/kernel_summary.csv`。

表 2: 三次 K-best 的均值 ± 样本标准差。

内核	getpid	大块写	大块 munmap
3.6.10	0±0 ns	1.42±0.12 ms	220.4±190.9 us
3.7.9	422±61 ns	2.63±1.18 ms	273.1±6.7 us
3.8.10	317±44 ns	1.91±0.44 ms	403.2±168.1 us
3.9.8	295±18 ns	2.70±1.34 ms	459.6±272.4 us
3.10.10	921±23 ns	2.26±0.24 ms	329.0±50.8 us
3.11.6	325±43 ns	2.04±0.42 ms	379.1±58.3 us
3.12.8	1.0±0.1 us	2.65±1.66 ms	350.1±61.8 us
3.13.7	1.1±0.2 us	1.63±0.07 ms	321.0±10.7 us
3.14.6	878±26 ns	2.32±0.15 ms	373.3±48.1 us
3.15.8	998±126 ns	2.38±0.56 ms	340.3±87.3 us
3.16.3	322±41 ns	1.09±0.46 ms	178.7±83.5 us
3.17.6	238±75 ns	1.54±0.68 ms	218.6±8.5 us

内核	getpid	大块写	大块 munmap
3.18.6	293±37 ns	2.93±1.42 ms	228.9±29.7 us
3.19.3	253±6 ns	2.08±0.40 ms	200.3±0.5 us
4.0.5	245±0 ns	1.66±0.28 ms	199.7±3.5 us
4.1.6	255±6 ns	1.95±0.50 ms	193.6±8.1 us
4.2.5	248±9 ns	1.37±0.57 ms	161.7±55.2 us
4.3.3	224±93 ns	1.67±0.55 ms	157.4±54.9 us
4.4.5	238±8 ns	1.65±0.10 ms	198.6±5.9 us
4.5.4	189±0 ns	1.59±0.15 ms	166.4±50.7 us
4.6.4	117±62 ns	1.67±0.94 ms	184.9±87.7 us
4.7.6	192±1 ns	4.24±2.97 ms	334.2±100.0 us
4.8.14	225±29 ns	2.87±1.62 ms	210.6±0.5 us
4.9.11	203±85 ns	3.72±1.20 ms	480.8±415.3 us
4.10.13	197±6 ns	1.71±0.15 ms	206.9±3.8 us
4.11.8	212±28 ns	1.63±0.40 ms	335.8±236.1 us
4.12.10	195±9 ns	1.58±0.27 ms	247.3±58.3 us
4.13.12	251±10 ns	1.90±0.73 ms	211.4±2.6 us
4.14.15	535±12 ns	1.77±0.21 ms	206.0±6.4 us
4.15.15	248±30 ns	2.55±0.64 ms	212.3±39.9 us
4.16.13	100±0 ns	657.6±46.5 us	83.6±0.8 us
4.17.14	100±1 ns	611.1±12.2 us	84.0±1.0 us
4.18.16	100±0 ns	671.0±17.4 us	79.5±0.7 us
4.20.13	230±44 ns	2.27±0.47 ms	192.7±3.9 us

4 分析

论断 1 看的是相对 4.0 的变化方向，不是绝对纳秒数。这里的绝对时间不是论文里的时间：客户机只有两个很慢的虚拟 CPU，磁盘是虚拟磁盘，迭代次数也更短，所以一个离群点就会挪动 K-best 窗口。表 1 里的标准差经常占均值的很大一部分。小于标准差的差值不能当作证据。下面的结论使用这条规则：只有当更晚内核的“均值 ± 标准差”区间整个落在 4.0.5 的区间之上时，才把它算作更慢。

按这条规则，论断 1 在这台客户机上**不成立**。627 项更晚内核的测量里，只有 271 项（43%）的均值高于 4.0.5，只有 117 项同时满足区间不重叠且均值更高。论文说的“大多数测试更慢，而且很多至少慢 50%”对不上这张表：627 项里只有 76 项（12%）的均值至少高 50%。拆到那 19 个更晚的内核上，getpid 有 5 个均值更高，有 9 个区间明显更快；fork 的更高均值个数是 0。论文把大块写当作变快的例外，它在 19 个内核里有 11 个均值更高，但只有 1 个区间超出了噪声。和 Linux 3.0 的比较是**无法判断**：3.0.7 到 3.5.4 从未连上 sshd，这台客户机上没有 3.0 基线。

论断 2 只碰到了 `nopti` 这一个开关。论文的 KPTI 结果需要一颗会为页表隔离付出代价的

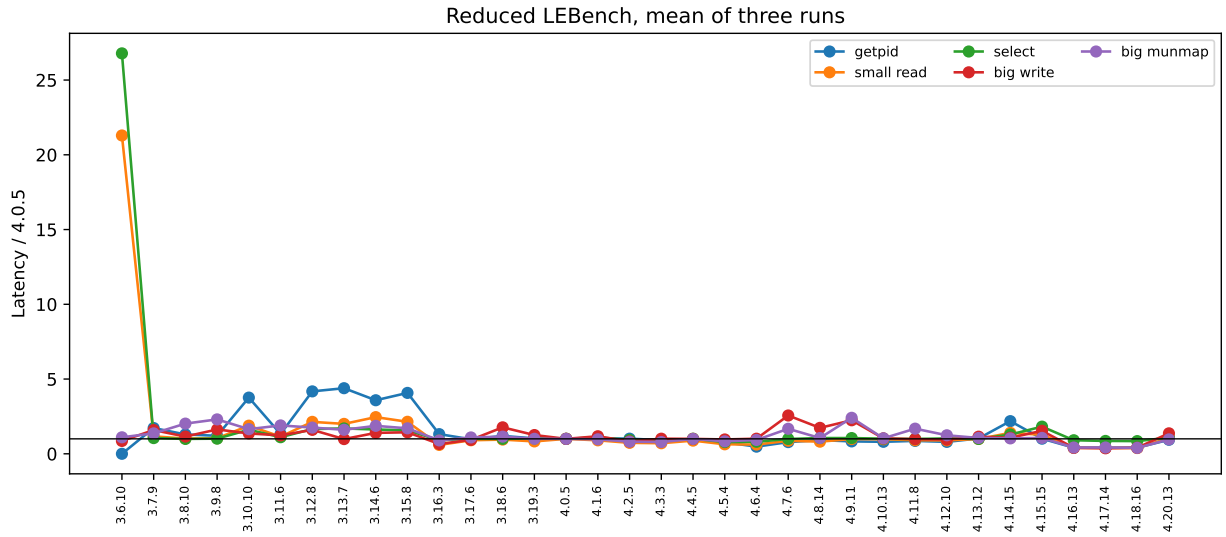


图 1: 延迟除以本客户机上 4.0.5 的结果。横线 1 是论文的基线内核。纵轴文字仍是英文，因为图由同一份汇总脚本画出。

CPU。i7-13700H 报告它不会。这个开关仍然改变了 `fork`，所以命令行不是空操作，但它没有复现出“KPTI 让 `getpid`、读和 recv 变慢”。论断 2 的这一部分在这里**不成立**。

论断 3 没有测。本仓库里没有 Redis、Apache 或 Nginx 的数字，所以应用层面的论断**无法判断**。

这次运行给不出原服务器上的热力图、十一项消融，以及应用程序的加速。到不了 sshd 的内核记在 `results/kernels/progress.log` 的启动失败里，不给它们填一个延迟。

5　扩展：把 getpid 改成批量计时

精简套件在两次 `clock_gettime` 之间只调用一次 `getpid`。在 4.0.5 上，空跑的 `ref` 是 156 ± 64 ns，`getpid` 是 245 ± 0 ns，计时器占了这个数字的很大一部分。扩展实验只改这一项。`course/OS_Eval_batched.c` 从精简源生成，把 `getpid` 改成一次计时 1,000 次调用再除以 1,000。K-best、`BASE_ITER`、编译选项和其它测试都不变。每个内核把新二进制跑了三次。日志在 `results/batching/<版本 >/run{1,2,3}.csv`。

表 3: 三次 K-best 的均值 ± 样本标准差。单次一列来自精简扫描，批量一列是上面这一处改动。

内核	单次 getpid	批量 getpid
4.0.5	245 ± 0 ns	129 ± 0 ns
4.4.0-210	314 ± 39 ns	127 ± 40 ns

在 4.0.5 上，批量之后每次调用是 129 ns，单次是 245 ns，两个区间不重叠。同一天这份二进

制的空跑时钟对是 132 ± 19 ns，和批量后的每次调用接近。在 4.4.0-210 上，单次均值是 314 ns，区间整个在 4.0.5 之上。批量之后两个均值是 127 ns 和 129 ns，区间重叠。这两个内核之间看起来的 `getpid` 变慢，在这一处改动之后没有了。批量二进制上的 `small read` 仍落在单次结果的波动里（4.0.5：813 ± 72 ns，对照 726 ± 73 ns），所以套件的其余部分没有跟着这次修改一起变。

6 另一份实现

仓库外面还有一棵从头重写的 LEBench。它是一组 Linux C 程序（`src/bench_*.c`），配一份结果表为空的中文报告。上面的数字没有用它。里面有用的是 K-best 的说明和操作列表。测量本身不能拿来替换 `OS_Eval`：

- 它用 `rdtsc` 计时，而且没有把指令流串行化。在这台 VirtualBox 客户机上，TSC 相对宿主机的偏移不稳定，所以作者的代码用 `clock_gettime`。
- `select` 在计时区间里分配并填满描述符位图，数字里包含了分配器的开销。
- 上下文切换是一整段循环再除以迭代次数。这是平均值，不是论文里反复试验的 K-best，也没有把优先级调到 −20。
- 缺页样本会摸每一页，再除以页数。第一次缺页时，fault-around 会把相邻页一起填上，所以同一次样本里后面的触摸常常不是缺页。`OS_Eval` 只给一次触摸计时。
- 迭代次数、空系统调用的选法，以及没有第三次进程运行，都和论文、也和课程要求不一样。
- 它只在 Windows 上对着占位头文件编译过，没有在 Linux 上执行。那份报告自己也是这么写的。

让课程这棵树更接近论文的改动在 `course/` 和 `scripts/vm_kernel_sweep.py`：调用作者的函数，保留 K-best，重复三次，建好 `OS_Eval` 拼到 `LEBENCH_DIR` 后面的 Unix 套接字目录，并用 `grub-reboot` 启动，而不是把 GRUB 装到一个分区上。

7 结论

这次复现是部分完成的。作者列出的 41 个内核里，34 个把精简基准跑了三次。在这些日志上，“4.0 之后大多数核心操作变慢了”这一条不成立；在一颗不受 Meltdown 影响的 CPU 上，`nopti` 也没有复现出 KPTI 变慢。在 4.0.5 和 4.4.0-210 上把 `getpid` 改成批量计时之后，报告的每次调用大约少了一半，两个内核之间的变慢也不见了，所以原来那一行主要是计时器。应用程序一节和十一项原因的消融没有复现。实际的教训是：把微基准抄到一个新计时器上，以及一个装得上却启动不了的内核镜像，都会安静地失败。前者仍然会打出一个数字，后者则根本走不到打印数字的那一步。日志必须能看出发生的是哪一种。

贡献说明

邹浩宇（5260000157）导入了 `6aa7801` 的 LEBench，安装了 Ubuntu 16.04.6 客户机，编写了精简运行程序和重启驱动，收集了 `results/kernels/` 下的日志，测量了 `results/batching/` 下的批量 `getpid`，并撰写了本报告和幻灯片。这项工作的提交说明使用该学号和姓名。

AI 使用说明

Cursor 里的 AI 编程助手起草了运行程序、重启脚本、本报告和幻灯片，并操作了虚拟机。内核包、基准程序源代码和延迟数字来自 LEBench 作者以及客户机日志。助手没有编造测量值。

参考文献

[1] Xiang (Jenny) Ren, Kirk Rodrigues, Luyuan Chen, Camilo Vega, Michael Stumm, and Ding Yuan. An Analysis of Performance Evolution of Linux's Core Operations. In *Proceedings of SOSP*, 2019. `https://doi.org/10.1145/3341301.3359640`